

2.1.1 Trace Tables

ٹریس ٹیبل الگورتھم کا کاغذی اجرا (Dry Run) ہے۔

CS XII

Chapter 2: Computational Thinking & Algorithms

Lecture 02 of 24

★ Golden

2.1.1 Trace Tables ★ Golden

LEARN IT

A **trace table** is a technique used to monitor how the values of variables change during the execution of an algorithm. It is a **paper execution** of the code — a manual walkthrough, also called a **dry run**. You act as the computer and fill in one row after another.

Purpose of trace tables

- **Dry running:** execute the algorithm line by line by hand.
- **Debugging:** find logical errors where the algorithm runs but the output is wrong.
- **Validation:** check that the algorithm does what it is intended to do.

Components of a trace table

- **Line / step number** — which line is being executed.
- **Variables** — one column for every variable.
- **Input / output** — user inputs and printed results.
- **Condition check** — True/False for IF, WHILE, FOR, and similar tests.

WORKED EXAMPLE

Worked example: soldiers who meet the height rule. To qualify for training, a soldier's height must be between **1.57 and 1.91 metres, inclusive**.

Inputs: all soldiers' heights. Output: the count of eligible soldiers.

Step 1: Start
Step 2: SET Total = 0
Step 3: Set AllHeights = [1.91, 2.01, 1.87, 1.57, 1.48]
Step 4: FOR EACH height FROM AllHeights DO
Step 5: IF height >= 1.57 AND height <= 1.91 THEN
Step 6: SET Total = Total + 1
Step 7: END IF
Step 8: END FOR EACH
Step 9: Print Total
Step 10: End

Inclusive means the two boundary values **count**: 1.57 and 1.91 are eligible.

Step	Total	Height	AllHeights (input)	Height between 1.57 and 1.91?	Output
2	0				
3	0		1.91, 2.01, 1.87, 1.57, 1.48		
4	0	1.91			
5	0	1.91		True	
6	1	1.91			
4	1	2.01			
5	1	2.01		False (skip Step 6)	
4	1	1.87			
5	1	1.87		True	
6	2	1.87			
4	2	1.57			
5	2	1.57		True (boundary)	
6	3	1.57			
4	3	1.48			
5	3	1.48		False (skip Step 6)	
9	3				3

Eligible: 1.91, 1.87, 1.57. Not eligible: 2.01 (above 1.91) and 1.48 (below 1.57). Printed Total = 3.

```
AllHeights = [1.91, 2.01, 1.87, 1.57, 1.48]
Total = 0
for height in AllHeights:
    if height >= 1.57 and height <= 1.91:
        Total = Total + 1
print(Total)
```

OUTPUT
3

REMEMBER IT

How to fill a trace table without getting lost. Write a new row every time a variable **changes** or a condition is **tested**. Copy unchanged values into the new row so you never guess. Circle the final output.

Mnemonic: **V-I-C-O** — Variables, Input, Condition, Output. If a column is missing, the table is incomplete.

COMMON MISTAKE

Mistake 1: treating 1.57 or 1.91 as out of range. The words "inclusive" and the operators $\geq$ and $\leq$ mean the boundaries **are allowed**.

Mistake 2: updating Total on a False condition. If the IF is False, skip the SET line and leave Total unchanged.

Mistake 3: writing only the last Total. Examiners award marks for the **changing values in the table**, not only for the printed 3.

اردو میں سمجھیں

ٹریس ٹیبل الگورتھم کا کاغذی اجرا (Dry Run) ہے۔ ہر قطار میں لکیر نمبر، متغیرات کی نئی قیمتیں، ان پٹ/آؤٹ پٹ اور شرط کا نتیجہ (True/False) لکھا جاتا ہے۔ فوجیوں کی مثال میں قد 1.57 سے 1.91 میٹر تک ہونا چاہیے، حدود سمیت؛ پانچ قدوں میں سے تین اہل ہیں اس لیے آؤٹ پٹ 3 ہے۔

EXAM TIP

Very common long question: "Dry-run the given algorithm using a trace table and state the output." Draw columns for **every** variable plus a condition column. Show the loop once per item. Boundary values ($\geq$, $\leq$, empty list) are favourite traps. Short questions ask: purpose of a trace table (dry run / debug / validate) and its components (line, variables, I/O, condition).

CHECK YOURSELF

1. List the four usual columns / components of a trace table.
2. In the soldier example, why is height 1.57 counted but 1.48 not counted?
3. What is printed, and how many times is Step 6 executed?

Answers: 1. Line/step number, variables, input/output, condition check. 2. 1.57 is the lower boundary (inclusive, so True); 1.48 is below 1.57 (False). 3. 3 is printed; Step 6 runs three times (for 1.91, 1.87 and 1.57).

Next lecture: 2.1.2 Stepwise Reasoning